

HighGround: full project audit

Updated Sun 11 Oct, ~09:00 IST (offline + navigation built and reviewed; judge #2; search fix; footage and rough cut). **One file with everything:** what we built, how it works, the tech stack (used and planned), every number we may quote (with its caveat), what is done and what is not, how to run it, the pitch, the 3-minute video, and what to do next. Written Sun 11 Oct 2026, ~00:45 IST, for the team. Numbers are copied from the model output files (`data/out/web/proof.json`, `runs.json`, `hospitals.json`, `parking.json`, `streets/index.json`, `graph/meta.json`) and the run logs. **If you quote a number, quote it as written here, with its caveat.**

Companion files: `docs/OFFLINE_AND_NAVIGATION_PLAN.md` (next features, sourced), `docs/LANDING_PAGE_CONTEXT.md` (landing page), `docs/DEMO.md` (generated demo streets), `docs/blog_draft.md`, `PROGRESS.md` (full engineering log), `CLAUDE.md` (original spec).

0. Status at a glance

Item	State
Deadline	Sun 11 Oct 2026, 20:00 IST (online submission; confirm on the wemake devs.org/aws/env form)
Flood model (16 storms) + validation + ANUGA cross-check	Done , v3 shipped (v4 tested and rejected by a rule written before the run)
Web app (all screens, replays, time-lapse, routing, hospitals, proof, about, assistant UI)	Done and working locally (dev :5173 , production build :4173)
AWS backend (SAM stack, 4 Lambdas, API, DynamoDB, SNS, Scheduler, Bedrock, Location, Amplify)	Code done, tested against mocked AWS (moto). Not deployed.
What blocks the deploy	The team's AWS profile <code>highground</code> does not exist on this laptop yet; <code>.env</code> still has the placeholder email; Bedrock model access not confirmed. See section 16
Live URL, real alert email, live assistant	After deploy
Demo video	Footage recording (opening done; search, navigate, timelapse, what-if, proof in progress). Rough cut builder <code>docs/make_rough_cut.py</code> → <code>review/p5/footage/rough_cut.mp4</code> + cut list <code>docs/VIDEO_CUT.md</code> , with two slots for the alert email and the assistant (need the deploy)
Blog	Draft in <code>docs/blog_draft.md</code> (numbers corrected tonight)
Landing page	Teammate; context in <code>docs/LANDING_PAGE_CONTEXT.md</code> + PDF
Offline mode + flood-safe navigation	Built Sun 11 Oct, 00:55–03:05 IST (commits 4119956, fdcfb10, a7bbd2f, c8add5b): Save for offline, battery saver map, help card, flood plan, Take me to dry ground with directions, live GPS directions and Preview the drive. Offline gate and route-rule tests pass; reviewer pass running. Plan: <code>docs/OFFLINE_AND_NAVIGATION_PLAN.md</code>

Item	State
Phase gates	P1 passed and tagged p1 . P2: mass balance, depth maps, Pallikaranai wet and T. Nagar mostly dry pass; "Velachery wetter than T. Nagar" fails (disclosed). P3 passed (ranking test). P4/P5 need the deploy. Tags p2/p3 not yet created

1. The problem and the pitch

One line: HighGround shows Chennai residents where floodwater will go tonight, street by street, and what to do about it.

The five questions it answers:

1. Will my street flood tonight? How deep, and when?
2. Where can I park my car on dry ground?
3. Which way can I drive there without crossing water, and by when must I leave?
4. Which hospitals get cut off?
5. Email me if this changes.

Why it matters (sourced, safe to say):

- Every monsoon, Chennai parks its cars on flyovers. In Cyclone Fengal (2024) the Velachery, Pallikaranai, Medavakkam and Mint flyovers were full of cars ([Deccan Herald](#)).
- The 2015 floods produced Rs 4,800 crore of insurance claims, mostly motor ([Business Standard](#)).
- In 2015, 18 ICU patients at MIOT died when floodwater reached the generator room ([CS Monitor/AP](#)).
- On 4 Dec 2023 (Cyclone Michaung), 712 of Chennai's 1,814 11 kV power feeders were switched off ([TNM](#)); on 5 Dec, 30% of the city's 42,747 mobile towers were down ([The Week/PTI](#)). This motivates the offline plan.

Positioning (from the spec, still valid): Chennai already has CFLOWS, a government flood warning system built for officials. HighGround is for the residents who still park their cars on the Velachery flyover every storm because nobody tells them where the water will go. We do not claim a better model than CFLOWS; we claim a tested one, built for citizens.

Positioning update (important, found tonight): the state's **Chennai Flood Monitor** (RTFF & SDSS, World Bank funded, reported operational since Oct 2025) is public, bilingual and *claims street-level inundation forecasts* for vulnerable areas (Velachery, Saidapet, Mudichur). **TN-Alert** (state app, 5 lakh+ installs) sends official rain and flood alerts in Tamil. So **never say we are the only street-level forecast**. Say what we add:

Official systems tell you a flood is coming. HighGround tells you what to do on your street: where to move your car and by when, which way to drive without crossing water, which hospitals you can still reach, and an email when that changes. And we show how well the model did on 2015, honestly.

Tagline options: "Where the water goes tonight, street by street." · "Know before the water does." · "Every monsoon, Chennai parks its cars on flyovers. Now you know where, and by when."

2. The event and how we score

- **Environmental Hacks**, event 02 of the **Bharat Builds Tour** (WeMakeDevs × AWS), **Heat and Water** track. Hybrid, Oct 8–11, 2026. Teams of 1–4. Must be built during the event, must use AWS and **show it in the video**, 3-minute video. Blog on AWS Builder Center for the blog prize.
- Repo: https://github.com/Divue/high_ground

Criterion	Our evidence
Idea and impact	A daily, personal decision (move the car, by when, which way) for a city that floods every year; hospitals cut off; offline mode for outages (planned)
Built on AWS	12 AWS services, each doing real work (section 6); open data read straight from the AWS Open Data registry; one SAM template
Design and usability	Full-screen 3D night city; plain words ("waist-deep", "too deep for scooters from 9 PM"); one orchestrated opening; amber means safety; red only for 112
Execution	A real 2D flood model (16 storms, mass conserved exactly), a second independent model as cross-check, honest held-out validation, in-browser routing over 206k road edges, ~57 fps
Demo video	Frame-perfect recorder for smooth 1080p footage; script in section 13

Earlier simulated-judge scores (Fri, before deploy and before the UX overhaul): idea 8, AWS 5 (≈8 once deployed), design 6, execution 7, demo readiness 5. Its top advice: **deploy and film the real alert email and a grounded assistant answer**. Still the single biggest lever.

3. The product (what a resident sees)

The whole app is one full-screen 3D night map of Chennai with floodwater on it.

Screen	What it does
Opening	Camera starts low over the Bay of Bengal at night and flies in to a Velachery street in ~5 s; a title line ("Chennai, 6 PM. Cyclone Michaung, 2023, replayed as if it were tonight."); rain starts; water rises; the answer card appears (~12 s after load)
Tonight (hero)	Type a street → the camera pulls up, travels across the city and descends to it ("Going to ..." with progress) → a pulse marks the street → the night replays from 6 PM and the water rises hour by hour → the answer card: depth in body words ("Waist-deep") plus cm, when it becomes too deep for scooters, a picture of an adult, a scooter and a car against the water line, "Move your car to X by 2 AM" , the nearest dry parking (mostly flyovers) with "1 more dry place nearby", an amber route that draws itself, and "Email me if this changes" . A sticky "In danger, call 112" line
Replay a storm	Tonight's live forecast is usually dry, so the app replays real storms as if tonight: Dec 2015 (with the reservoir release), Cyclone Michaung 2023 , Cyclone Fengal 2024 . When the forecast is dry, the card says so and offers a replay
Watch the whole storm	~30 s time-lapse of a replay on its real dates, with captions from the model files ("43 mm fell in this one hour", "Chembarambakkam starts releasing water", "most land under water"). Pause, resume, watch again, back to my street, progress bar
What if	Rainfall slider over the computed storms (50–400 mm in a day), mean or high tide; the city floods and drains; "% of the city too deep for scooters" and "hospitals cars cannot reach"
Hospitals	46 major hospitals; which are cut off from the main road network at each storm size; the dry route to each reachable one
Proof	"Did the model get 2015 right?" Swipe split-screen: model's 2015 flood vs streets residents reported; the honest result on one shared scale; ANUGA cross-check toggle; "For experts" sections

Screen	What it does
About the model	Limits in plain words; data credits; Animations on/off
Ask HighGround	Assistant on Bedrock (Strands Agents) that answers only from model tools; every number is checked against tool output and marked "from the model"; tells people in danger to call 112 first. Shows "offline" until deployed
#admin (hidden)	"Run and send alerts" for a chosen rainfall, to trigger a real alert email for the demo
Take me to dry ground (new)	On the answer card: On foot / Two-wheeler (default) / Car to Dry parking, a Hospital, High ground or a saved place. The route avoids streets the model expects to be 10 cm deep (on foot, two-wheeler) or 20 cm (car) at the hour you would reach them, closes underpasses in rain and keeps to one-way streets; it plans for the hour on the timeline (drag to replan). Shows the dashed usual way it avoids, step-by-step directions, and the honesty line. Start follows your GPS with one big instruction and voice; Preview the drive plays the route for a demo
Save for offline (new)	Keeps the app, the map, this street (and up to 2 more places, 3 km around each) for every storm, the road graph and the forecast on the phone. Offline, the app opens from the phone, says "Offline · forecast from 5:24 PM (6 h ago)", and the map switches to battery saver: flat, streets coloured by depth
Help numbers (new)	Tap-to-call 112 (red), 1913, 1070, 1077, 108, 101 and Minnagam 94987 94987 for fallen wires; "Send my location" as SMS / WhatsApp / share / copy
My flood plan (new)	The answer as an image and as plain text (depth, time, the decision, dry parking, the route and its streets, the nearest hospital cars can reach, numbers) to save or send to family; works with no app and no network

Interaction details: tap any street for its worst depth and "Make this my street"; play button on the timeline; map controls (zoom, compass/tilt, 3D/flat, slow spin, back to my street); buildings coloured by height; terrain colour-relief and hillshade; flowing water that runs downhill (flow map from the water-surface slope); screen-space rain; a "Play animations" offer for devices that ask for reduced motion (?motion=on|off , remembered).

Design language: Chennai at 2 AM mid-storm; the water is the brightest thing on screen. Background #050D12 , land #0A161C , text #D6E0E4 , shallow water #86E6EC , deep water #1E7BC6 , **sodium amber #F2A541 only for safety** (parking, safe route), **red #E5484D only for 112**. Type: Anek Latin (the depth readout widens and gets bolder with depth) and Hind. Sentence case, no jargon, no arrows on buttons, no glassmorphism grids.

4. How it works

4.1 Architecture

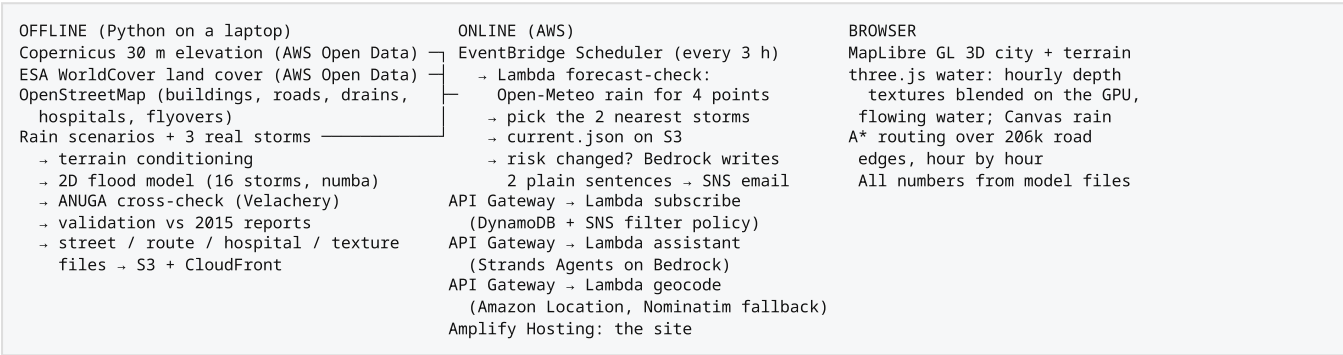


Diagram image: `docs/architecture.png` (also `docs/landing-assets/10_architecture.png`).

4.2 The flood model in plain words

1. **Ground.** The Copernicus GLO-30 elevation model (30 m) includes rooftops and trees. We removed **371,679** OpenStreetMap buildings, tree canopy (ESA WorldCover), bridges/flyovers and other tall objects, filled the gaps from surrounding ground, filtered residual clutter, restored the datum, carved waterways by type (rivers 2 m, canals 1.5 m, streams 0.6 m, drains 0.4 m), capped fake closed pits at 1 m, and put dense buildings back as +3 m walls water flows around. Grid $836 \times 1440 = \sim 1.2$ million 30 m cells, EPSG:32644 (UTM 44N).
2. **Water.** A 2D local-inertial shallow-water solver (Bates et al. 2010, the LISFLOOD-FP formulation) in Python/numba: adaptive timestep (CFL 0.5), Froude cap, donor-cell outflow limiter. **Mass error 0.000% on every run.** Rain is a source term; drains are one uniform capacity (10 mm/h, a stated assumption) on urban cells; infiltration on pervious land; sea held at mean or high tide; land edges are free outfalls except where the Adyar enters. ~ 15 min per storm on this 2-core laptop.
3. **Storms (16 runs).** 12 design storms (50, 100, 150, 200, 300, 400 mm in 24 h, front-loaded, each at mean and high tide) and real ones:
4. **1–2 Dec 2015**, 374 mm, run twice: rain only, and rain + the **Chembarambakkam reservoir release** (inflow at the Adyar's upstream edge, CAG/PWD audit timeline: 10,000 $\rightarrow$ 12,000 $\rightarrow$ 20,960 $\rightarrow$ 29,000 cusecs, 29,000 held 21 h; a documented assumption).
5. **Cyclone Michaung 2023**, 415 mm (IMD Meenambakkam).
6. **Cyclone Fengal 2024**, 114 mm (IMD bulletin).
7. Storm timing from ERA5 (via Open-Meteo), scaled to IMD totals.
8. **Outputs per storm:** max depth, time to 15 cm, hourly depth frames.
9. **Per street:** 168,156 street pieces, each with an hour-by-hour depth series, worst depth (p90 of its samples) and the hour it passes 15 cm; a "pre-wet" flag when the model already holds ≥ 5 cm there before the rain (low ground beside channels).
10. **Features:** a 162,067-node / 206,223-edge road graph with per-storm peak and hourly edge depths; **168 dry-parking candidates** (122 flyovers, 34 open grounds, 12 multi-storey); **46 major hospitals** (campus $\geq 4,000$ m² or 100+ beds); a 8,427-name local gazetteer.
11. **Cross-check (Stage B):** ANUGA (Geoscience Australia) on a 103,159-triangle mesh over Velachery and Pallikaranai, same terrain and 200 mm storm, first 14 h.
12. **Validation** against 2015 citizen reports on held-out wards (section 5).
13. **Export:** depth PNG frames (sea masked), terrain-RGB tiles, street JSON tiles (288 tiles of 2 km), routing binaries, `proof.json`, `runs.json`, `hospitals.json`, `parking.json`, `places.json`.

4.3 The live part (AWS)

- **forecast-check Lambda** (EventBridge Scheduler, every 3 h): Open-Meteo hourly rain for Velachery, T. Nagar, Anna Nagar, Tambaram $\rightarrow$ 24 h mean $\rightarrow$ the two nearest modelled storms and blend weights $\rightarrow$ `current.json` on S3 $\rightarrow$ compares each subscribed street's risk band with the last run $\rightarrow$ Bedrock writes two sentences from the model's numbers $\rightarrow$ SNS publish with message attribute `segment_id`. Pre-wet streets get no change alerts (they would flap).
- **subscribe Lambda:** email + street $\rightarrow$ DynamoDB $\rightarrow$ SNS email subscription with a filter policy on `segment_id` (the person confirms by email).
- **assistant Lambda:** Strands Agents SDK on Bedrock, six tools: `get_street_risk`, `find_dry_parking`, `safe_route`, `hospital_status`, `current_forecast`, `model_limits`. Every number in the reply is checked against tool outputs.

- **geocode Lambda:** Amazon Location Places v2 (biased to Chennai), falls back to OSM Nominatim and says which answered. Without the API the app searches its local gazetteer.

4.4 The browser

- MapLibre GL JS 6: Protomaps dark basemap from one PMTiles file, `raster-dem` terrain (terrain-RGB tiles), colour-relief + hillshade, extruded buildings coloured by height, sky and fog.
- three.js custom layer: one water mesh draped on the terrain; hourly depth textures cross-faded in the shader (`mix`); depth colour ramp, 30 cm line, fresnel, glints, sea mask, flow map; drawn without depth test so it never flickers by camera angle. PNG decoding in a Web Worker; render-on-demand map; flow animates ~30 fps only when zoomed in and recently touched.
- Rain: screen-space Canvas2D streaks in 3 parallax layers, following hourly rainfall.
- Routing: A* in the browser over the road graph; per-mode limits (cars 30 cm, two-wheelers 15 cm today); "leave by" = latest hour a dry route exists, using hourly edge depths; "no practical dry route" when the detour is >3× or >+4 km.
- Camera: three-phase travel (pull up, cross the city facing the target, descend), landing pulse, settling orbit; any touch cancels.

5. Numbers we may quote (with caveats)

5.1 Validation (the honest headline)

The test: on GCC wards **never used for tuning** (even-numbered wards; odd wards were the tuning half), pick a street residents reported flooded in Dec 2015 and one they did not. How often does the model put more water on the reported one? (A rank test that cannot be gamed by flooding more.)

Map	Score	95% range (resampling whole wards)
HighGround model (2015 rain + release)	53% (0.532)	51% to 55%
Coin toss / random map	50% (random map scored 49%)	
Low ground alone	47%	41% to 51%
Distance to the nearest canal or river	56% (better than the model)	53% to 60%

- **Verdict, word for word: "slightly better than chance."** Quote it together; never quote 53% alone.
- 3,200 reported street segments in the held-out wards (27,970 unreported); 7,894 reported segments citywide.
- **Rain vs river:** rain-driven streets 54%; river-driven streets (within 500 m of the Adyar or Cooum) **51% without the reservoir release, 53% with it:** the release matters by the river.
- Satellite (NRSC 2015) flood extent: the model covers 33% of it; a random map of the same size 31%; the lowest ground of the same size 21%.
- GCC's own 2015 flood hotspots (327 points): model 53%, low ground 55%, nearest channel 57%, random 49%.
- Ward level: weak (Spearman 0.11 over 100 wards, not significant).
- Drain capacity: 10 mm/h is a **stated assumption:** on the tuning half, every value from 0 to 30 mm/h scored the same within error bars.
- **Withdrawn metric (a good story):** our first score ("share of reported streets the model floods") said 33% vs 19% for an elevation map. A reviewer showed a random speckle of the same size scores 50% on it: it rewards patchy maps. We withdrew it.

5.2 Cross-check with a second model

ANUGA vs our solver, Velachery box, 200 mm storm, first 14 h: **they agree on 73% of the cells where either model has ≥ 15 cm** (critical success index); depth correlation 0.90; 94% of all cells agree (inflated by shared dry cells, so lead with 73%). Same terrain, so this checks the arithmetic, not the terrain.

5.3 Storm outcomes

Storm	Rain	City land ≥ 15 cm ("too deep for scooters")	Hospitals cut off (of 46)
50 mm design	50 mm / 24 h	0.9%	—
100 mm design	100 mm / 24 h	2.4%	0
150 mm design	150 mm / 24 h	9.5%	—
200 mm design	200 mm / 24 h	17%	9
300 mm design	300 mm / 24 h	25%	15
400 mm design	400 mm / 24 h	30%	16
Dec 2015, rain only	374 mm	29%	15
Dec 2015, rain + release	374 mm	29%	16
Cyclone Michaung 2023	415 mm	23%	15
Cyclone Fengal 2024	114 mm	3.6%	0

(Mean tide for design storms; high tide differs by under 1 point. "—" = not looked up for this file.) Smaller storms are probably overstated: drains are one uniform assumption.

5.4 Scale and performance

~1.2 million 30 m cells · 371,679 buildings removed from the terrain · 16 storm runs · mass error 0.000% · 168,156 street pieces · 206,223 road edges · 168 dry-parking candidates (122 flyovers) · 46 hospitals · 8,427 searchable names · ~57 fps measured on an integrated-GPU laptop with the water flowing · answer card ~12 s after load · routing ~11 ms median per city-wide trip (measured on the real graph).

6. Tech stack

6.1 Used now

Layer	Tools (versions as installed)
Frontend	Vite 8.3.4 , React 19.3 , TypeScript , MapLibre GL JS 6.13.0 (3D terrain, extrusions, colour-relief, hillshade), three.js 0.186.1 (custom water layer, GLSL), PMTiles 4.5.0 + Protomaps basemaps 5.7.2 , Web Workers, Canvas 2D
Model pipeline	Python 3.11 , numba (solver), NumPy, SciPy, rasterio/GDAL, GeoPandas, pyproj, scikit-image, pandas, pyosmium; ANUGA 4.0.1 (separate env)
AWS	S3 , CloudFront (OAC, CORS), Amplify Hosting , Lambda (Python 3.12 $\times 4$), API Gateway (HTTP API), DynamoDB , SNS (filter policy per street), EventBridge Scheduler (<code>rate(3 hours)</code>), Bedrock (Claude Sonnet: alerts + assistant), Strands Agents SDK (AWS open source), Amazon Location Service (Places v2), AWS SAM , AWS Open Data Registry (Copernicus DEM, ESA WorldCover), AWS Budgets (\$10 alarm)

Layer	Tools (versions as installed)
Testing / tooling	Playwright (end-to-end hero test, frame-by-frame review, frame-perfect footage with a virtual clock), moto (mocked AWS for Lambda tests), ffmpeg (OpenH264), Miniforge/conda, nvm/Node 24
Data / services	OpenStreetMap (Tamil Nadu extract), Open-Meteo forecast + ERA5, IMD rainfall totals, OpenCity datasets (2015 crowd-sourced flood reports, GCC hazard zones, NRSC 2015 inundation, GCC wards, GCC flood hotspots and stagnation points)
AI agents during the build	Claude Code with reviewer subagents: model-reviewer, design-critic, qa-tester, judge; research agents for creative ideas, offline, navigation and Chennai ground truth

6.2 Added for offline and navigation (built Sun 11 Oct)

Purpose	Tool
Service worker / PWA	<code>vite-plugin-pwa</code> 1.3.0 (injectManifest) + Workbox 7.4.1
Offline basemap	basemap cut at zoom 14 with the <code>pmtiles</code> Python package (7.1 MB, used online too); the service worker answers its byte ranges from the saved copy (<code>workbox-range-requests</code>)
Offline storage	Cache API (<code>hg-pack-<version></code> , <code>hg-seen</code> with <code>workbox-expiration</code>) + <code>navigator.storage.persist()</code> / <code>estimate()</code>
Offline search	the existing local place list (8,427 names) works offline; typo-tolerant search not added
Self-hosted fonts	@fontsource-variable/anek-latin, @fontsource/hind 5.3.0; Protomaps glyphs (Noto Sans) and sprites in <code>web/public/basemap-assets/</code>
Navigation	time-dependent search in a Web Worker (<code>lib/nav.ts</code>), own directions generator (<code>lib/directions.ts</code> , GraphHopper turn thresholds), OSM one-way/roundabout/tunnel flags (<code>pipeline/05b_navigation.py</code>), Geolocation <code>watchPosition</code> , Screen Wake Lock, Web Speech <code>speechSynthesis</code> , Web Share (files), canvas image of the plan
AWS additions	offline pack files on S3/CloudFront with a version manifest; routing binaries gzipped at rest; Amplify custom headers (service worker never cached, assets immutable) and a fixed rewrite rule. Later: SACHET CAP official warnings via Lambda, Web Push (VAPID in SSM), AWS End User Messaging SMS (needs TRAI DLT)

7. Honesty: limits, claims to make, claims to avoid

Limits (must appear in the app, video or page):

- 30 m satellite terrain with buildings and trees removed by approximation: kerbs, gates, culverts and small dips are invisible.
- Drains are one uniform capacity (10 mm/h), an assumption, not a drain network. Smaller storms are probably overstated.
- The model does not reproduce GCC's finding that Velachery floods far more than T. Nagar; its flooding follows small hollows in the 30 m terrain. Some places that flooded (e.g. Pallikaranai and Kotturpuram streets) read dry; don't use them as demo addresses.
- The 2015 reservoir release is modelled from the CAG audit timeline; other tanks are not included.
- No storm surge; rain is uniform across the city in each scenario. Some low ground beside channels holds water before the storm starts (flagged on the card).

- Underpasses are invisible to 30 m terrain.
- English only (about 18.5% of Tamil Nadu residents speak English; official tools are Tamil-first).
- **Not an official warning. Follow GCC, IMD and TNSDMA advisories. In danger, call 112.**

Claims to make: street-by-street answers in plain words; tested honestly on held-out wards, and we show the modest result; an independent second model agrees on 73% of the flooded area; built for residents; AWS-native; open data and open-source tools; email alerts (SMS coming: Indian SMS needs TRAI DLT registration).

Claims to avoid: "accurate", "predicts exactly", "official", "better than CFLOWS", "the only street-level forecast", "real-time flood model" (the model is precomputed; the forecast check is live), "AI predicts floods" (the AI only explains the model's numbers), "calibrated", "safe route" (say "avoids streets our model expects to flood"), flyover parking is "allowed", SMS alerts as working, any validation number without its caveat. For offline (if built): no "works on every phone", no "live updates during an outage".

8. What fought back (for the blog and the video)

1. **Satellite elevation is full of fake lakes.** Closed hollows in the 30 m data could hold 124 million m³, about 70% of a whole 200 mm storm. Fixes: cap artificial hollows at 1 m, carve drains by type (a street drain is not the Adyar), free-outfall edges so valleys drain.
2. **Channels leaked before the rain.** Filling rivers to their spill level flooded ~2% of the city at 6 PM in every storm. Caught while picking demo streets; fixed (start at the measured water surface); everything rerun.
3. **Our first validation metric was wrong, and a reviewer caught it.** It rewarded patchy maps (random speckle scored higher than the model). We withdrew it, switched to the ranking test, and stopped calling the drains "calibrated".
4. **A 2-core laptop.** First solver: 35 min per storm. Vectorised float32 kernels and a Halley-iteration cube root: ~11 min. A review then found sloshing at the original timestep (peaks inflated up to 1.6 m); a halving test fixed CFL at 0.5 (~15 min per storm).
5. **The model's edges leaked.** A third of the 2015 release ran out of the west edge; at high tide the sea ran through Ennore. Both fixed.
6. **The sea drawn as floodwater** in high-tide frames. Masked explicitly in export and shader.
7. **A better terrain (v4) that we did not ship.** We wrote the adoption rule before the run; v4 missed one condition (flooded land in GCC moderate-plus zones 23.7% vs 24.4%), so v3 stayed. Pre-registration over wishful thinking.
8. **Frontend lag** turned out to be rain forcing full-map redraws and PNG decoding on the main thread; fixed with a separate rain canvas, render-on-demand, and a decode worker.
9. **"Nothing animates"** on a teammate's laptop: the desktop had animations turned off, so the browser asked for reduced motion and the app skipped everything. Now it offers "Play animations" and remembers the choice.

9. Reviews and QA (process)

After every piece of work a reviewer agent ran, and every blocker and major was fixed:

- **model-reviewer** (hydrology): withdrew the gameable metric; fixed the release leak, sloshing (CFL 0.5), the sea short circuit, the pre-storm mask hiding deep riverside streets, and "leave by" using

display textures instead of model-grid depths; insisted on honest wording (CSI, ward bootstrap) and disclosure of the Velachery/T. Nagar failure.

- **design-critic**: first 5 seconds, water "draining then rising", contour clutter, number vs hour mismatch, email above the fold, deep-water colour, time-lapse captions, What-if range, basemap noise, mobile layout.
- **qa-tester**: terrain never loaded (URL encoding), crash on an empty scenario, rejected hospitals layer, a leave-by race on storm switch, coastal roads reading a constant 50 cm at high tide, wording.
- **judge**: scores and the top suggestion (deploy and film the real email and assistant; time-aware routing was built in response).
- Tests: Playwright hero flow 9/9 OK; storm switch, time-lapse, leave-by, reduced motion, production preview, camera angles; Lambda flow subscribe → forecast-check → SNS → filtered delivery passes against moto.

10. Repository map

CLAUDE.md	original spec (read first)
PROGRESS.md	engineering log, decisions, FOR THE USER steps
README.md	public overview + run instructions
pipeline/	
00_fetch.py	DEM tiles, OSM, WorldCover, validation KMLs, ERA5 timing
01_condition.py	bare-earth terrain, buildings, waterway burn, land cover
02_fast_model.py	local-inertial 2D solver (numba)
03_anuga_velachery.py	ANUGA cross-check
04_streets.py	per-street depth series, worst depth, time to 15 cm, pre-wet flag
05_features.py	road graph + edge depths (peak, hourly), parking, hospitals, gazetteer
06_validate.py	held-out ranking test, baselines, NRSC/GCC checks -> proof.json
07_export_tiles.py	depth PNG frames, terrain-RGB, runs.json, upload layout
gates.py	scripted phase gates (p1, p2, p3)
config.yaml	bbox, CRS, storms, release hydrograph, thresholds
run_all.sh, post_runs.sh, v4_chain.sh, v4_decision.py, common.py, scoring.py	
infra/	
template.yaml	SAM: S3, CloudFront, DynamoDB, SNS, HTTP API, 4 Lambdas, Scheduler, Amplify
deploy.sh	one-command deploy (budget alarm, Bedrock model pick, sam deploy, upload, Amplify)
functions/{subscribe,forecast_check,assistant,geocode}/app.py	
layers/common/python/hg.py	shared: nearest street, blend, risk bands, parking, hospitals, A*, limits
layers/strands/	Strands Agents SDK layer (py3.12 manylinux)
tests/test_lambdas_moto.py	
web/	
src/App.tsx	screens, opening, motion offer
src/map/	MapView (camera, controls, peek, pulse, orbit, drawLine), style (night palette)
src/water/	WaterLayer (three.js), shaders, RainOverlay
src/screens/	Tonight, WhatIf, Proof, Hospitals, About
src/ui/	Search, Timeline, Timelapse, DepthGlyph, Legend, Readout, Subscribe, Assistant
src/lib/	data (fetch + decode worker), routing (A*, leave-by), streets, scenario, words, geo
tests/	Playwright: hero, record_frames (footage), storm_switch, timelapse, angles, ...
data/out/web/	everything the site serves (symlinked as web/public/data)
docs/	this audit, offline/navigation plan, landing context (+PDF), demo guide, blog draft, architecture.png, landing-assets/, ideas/creative_research.md
review/	screenshots and reports per phase (p1, p2, p5, p5-dev, qa, design, model-review, judge)

11. How to run

```
# web, local (data already built)
cd web && npm install
npx vite --port 5173 # dev
npx vite build && npx vite preview --port 4173 # production build (use this for recording)
# demo links
http://127.0.0.1:4173/?replay=michaung2023 # Michaung replay
http://127.0.0.1:4173/?replay=michaung2023&motion=on # if the device reduces motion

# tests
node tests/hero.mjs http://127.0.0.1:4173/ # hero flow, screenshots to review/p5
source pipeline/env.sh && python infra/tests/test_lambdas_moto.py # Lambda flow vs moto

# model (long; run under systemd-inhibit so the laptop does not sleep)
source pipeline/env.sh
python pipeline/00_fetch.py && python pipeline/01_condition.py
systemd-inhibit --what=sleep pipeline/run_all.sh # 16 runs, hours
python pipeline/06_validate.py && python pipeline/04_streets.py && python pipeline/05_features.py && python pipeline/07_export_tiles.py
python pipeline/gates.py p1|p2|p3
```

```
# AWS (after the FOR THE USER steps)
infra/deploy.sh
```

Gotcha: after swapping `data/out/web`, restart Vite (its public-file list goes stale and `/data/*` returns `index.html`).

12. Pitch (for a deck, the landing page or a 60-second talk)

1. **Hook:** "Every monsoon, Chennai parks its cars on flyovers, because nobody tells residents where the water will go."
2. **Problem:** official systems warn that a flood is coming; residents still have to guess what that means for their street, their car, their route and their hospital.
3. **Solution:** type your street; watch tonight's storm rise on a 3D model of the city; get a plain answer: how deep, when it gets too deep for scooters, where to move the car and by when, the way there without crossing water, which hospitals you can still reach, and an email if it changes.
4. **How:** a real 2D flood model of all of Greater Chennai on open data from AWS, 16 storms precomputed, a live forecast check on AWS every 3 hours, an assistant that can only quote the model.
5. **Proof, honestly:** tested on wards we never tuned on against 2015 reports: slightly better than chance at street level (53% vs 50%), and we show that on screen; a second independent model agrees on 73% of the flooded area.
6. **Built on AWS:** S3, CloudFront, Amplify, Lambda, API Gateway, DynamoDB, SNS, EventBridge Scheduler, Bedrock, Strands Agents, Amazon Location, SAM, AWS Open Data.
7. **Next:** offline mode for power and network outages (towers down for days in 2015 and 2023), flood-safe navigation on foot / two-wheeler / car, Tamil, SMS once DLT-registered, better terrain.

13. The 3-minute video

Script (spec timings, updated):

Time	Shot	Voice-over / caption
0:00–0:15	Still of cars on the Velachery flyover (footage-style)	"Every monsoon, Chennai parks its cars on flyovers, because nobody tells residents where the water will go."
0:15–1:15	Opening flight from the Bay of Bengal → Michaung replay → search a Velachery street (pick from <code>docs/DEMO.md</code>) → water rises → answer card → "Move your car to X by ..." → amber route draws → Email me if this changes → the real SNS email arriving (trigger from <code>#admin</code> → Run and send alerts, 300 mm)	Plain description of each step; name Amazon Location (search), S3/CloudFront (model data), SNS + Bedrock (the email)
1:15–1:50	What if: drag to 400 mm, the city floods; Hospitals at 300 mm (15 of 46 cut off)	"At 300 mm, 15 of 46 major hospitals can't be reached by car."
1:50–2:30	Proof split-screen; "53% vs a coin toss's 50%"; one sentence on held-out wards; ANUGA 73%	Say the verdict word for word: "slightly better than chance"; "we show you that"
2:30–3:00	Architecture diagram (name every AWS service); two limits from About; close on What if dragged back to 50 mm (water recedes)	"Not an official warning. In danger, call 112."

New beat (built; fit it into 1:15–1:50 or trim What if): "When the power goes, the towers go." Network off (airplane mode on a real Android phone, or DevTools offline on the laptop): HighGround still opens, "Offline", the battery-saver map with streets coloured by depth, **Take me to dry ground · Two-wheeler** draws the amber route around the water with directions, **Preview the drive** plays it, **My flood plan** shared as an image. Caption: "Saved before the storm. Works without internet." Facts to show (sourced in the plan): 712 of 1,814 power feeders off on 4 Dec 2023; 30% of 42,747 towers down on 5 Dec 2023.

Footage commands (smooth 1080p/30 fps, virtual clock, waits for tiles):

```
cd web && npx vite build && npx vite preview --port 4173 &
WARM=1 node tests/record_frames.mjs opening http://127.0.0.1:4173/ ../review/p5/footage
WARM=1 node tests/record_frames.mjs search http://127.0.0.1:4173/ ../review/p5/footage
WARM=1 node tests/record_frames.mjs timelapse http://127.0.0.1:4173/ ../review/p5/footage
WARM=1 node tests/record_frames.mjs whatif http://127.0.0.1:4173/ ../review/p5/footage
WARM=1 node tests/record_frames.mjs proof http://127.0.0.1:4173/ ../review/p5/footage
```

About 30 s of wall time per second of footage; run under `systemd-inhibit --what=sleep`. Output: `review/p5/footage/<shot>.mp4` plus PNG masters. Still images ready now: `docs/landing-assets/01...10`.

Demo streets: `docs/DEMO.md` lists Velachery streets that flood in the Michaung replay (e.g. Dhandeeswaram Nagar 8th Cross Road, 118 cm). Check each in the search box before filming (search may resolve a different segment). Avoid Pallikaranai/Kotturpuram (model reads dry).

14. Offline mode and flood-safe navigation (built Sun 11 Oct; plan with

SOURCES: `docs/OFFLINE_AND_NAVIGATION_PLAN.md`)

What shipped, measured: offline gate (save → network off → reload → card, battery-saver map, plan image, help, offline search, offline route) passes with 0 page errors; route rules hold on 36 random trips over 3 storms × 3 modes × 3 destination kinds (0 edges over the limit at the reached hour, 0 one-way violations, 0 underpasses), median 33 ms per plan; live directions follow emulated GPS and re-plan after 3 off-route fixes. Stored size after saving one place is about 83 MB (street files are stored uncompressed). Not built: rainfall ladder for an old forecast, typo-tolerant search, Web Push, relief centres (needs geocoding of the GCC PDF), SACHET warnings.

- **Help card** (zero data): tap-to-call 112 (red), 1913, 1070, 1077, 108, 101, Minnagam 94987 94987; "Send my location" pre-filled SMS/WhatsApp.
- **My flood plan** card: image + text, shared via the phone's share sheet (works phone to phone without internet).
- **Save for offline:** ~18 MB (app, fonts, zoom-14 basemap 7.1 MB, 3 km of street answers for all 16 storms 2.0 MB, road graph 5.8 MB, edge depths); forecast updates are a 4 KB file; honest "forecast from 6 PM (5 h ago)"; rainfall ladder when stale.
- **Take me to dry ground:** foot / two-wheeler (default) / car; blocks at 10 / 10 / 20 cm; one-hour look-ahead; underpasses blocked in rain; one-way respected; prefers main roads; turn list; later live GPS turn-by-turn.
- **Build slices:** Slice 1 offline (~6 h), Slice 2 navigation with turn list (~7 h), Slice 3 live GPS (~5 h). Deploy and the video come first.

15. Open issues and to-do (owner)

#	Item	Owner
1	Create AWS profile <code>highground</code> (us-east-1), real <code>ALERT_TEST_EMAIL</code> in <code>.env</code> , Bedrock Claude Sonnet access	User
2	<code>infra/deploy.sh</code> → live URL; subscribe once, click the SNS confirmation link; <code>#admin</code> → Run and send alerts for the real email shot	Claude + user
3	Record the 1080p footage (section 13): in progress, then the rough cut	Claude
4	Edit the video; voice-over; submit	Teammate 1
5	Blog on AWS Builder Center (from <code>docs/blog_draft.md</code>); link in submission	Teammate 1
6	Landing page (context PDF already shared) — apply the positioning update in section 1	Teammate
7	~~Verify demo streets in the search box~~ done: <code>docs/DEMO.md</code> lists only verified streets (search now answers a named street with its deepest stretch)	Claude
8	Judge pass at the end; tag <code>p2</code> (with the documented place failure) and <code>p3</code>	Claude
9	README and blog: add the latest UX work (motion offer, rain, flow, camera)	Claude
10	Minor: assistant shows a raw error if Bedrock fails; mobile hides the timeline	Claude
11	Offline + navigation built; fix reviewer findings; film the new beat	Claude
12	Student verification on AWS Builder Center (eligibility)	User

16. FOR THE USER (only you can do these)

1. `aws configure --profile highground` → access keys for the team's AWS account, region `us-east-1`. (Checked tonight: the profile does not exist yet.)
2. Put your real inbox in `.env`: `ALERT_TEST_EMAIL=you@yourdomain` (still the placeholder).
3. AWS console (us-east-1) → Amazon Bedrock → Model access → enable an Anthropic Claude Sonnet model.
4. Then run `infra/deploy.sh` (or ask Claude). It creates the \$10 budget alarm, deploys the stack, uploads ~200 MB of model outputs, builds and deploys the site, runs the first forecast check, prints the live URL; the admin token for `#admin` is in `data/admin_token`.
5. Subscribe once from the live site with that email and **click the AWS Notifications confirmation link**.
6. Verify student status on AWS Builder Center.

17. Attribution lines (required wherever data or the map appears)

- Elevation: Copernicus DEM GLO-30, © DLR e.V. 2010–2014 and © Airbus Defence and Space GmbH 2014–2018, provided under COPERNICUS by the European Union and ESA; all rights reserved. Read from the AWS Open Data registry.
- Land cover: ESA WorldCover 2021 (CC BY 4.0), © ESA WorldCover project / contains modified Copernicus Sentinel data (2021), from the AWS Open Data registry.
- Map data: © OpenStreetMap contributors (ODbL); basemap by Protomaps.
- Rainfall forecasts and storm timing: Open-Meteo (CC BY 4.0), including ERA5 reanalysis (Copernicus Climate Change Service).
- 2015 flood reports: OpenCity, osm-in/flood-map contributors. Flood hazard zones, wards, hotspots: Greater Chennai Corporation via OpenCity. 2015 inundation extent: NRSC via OpenCity.
- Cross-check model: ANUGA, Geoscience Australia.

18. Glossary (for writing copy)

- **CFLOWS:** Chennai's flood forecasting decision tool for officials (NCCR).
- **Chennai Flood Monitor (RTFF & SDSS):** the state's public flood forecasting portal (since Oct 2025).
- **TN-Alert:** Tamil Nadu's official alert app.
- **Too deep for scooters:** 15 cm; **cars stall:** about 30 cm (the app's display thresholds).
- **Replay:** a real past storm run through the model and shown as if it started at 6 PM tonight.
- **Leave by / move your car by:** the latest hour a route to dry parking still avoids water deeper than the vehicle's limit.
- **Held-out wards:** the even-numbered GCC wards, never used for tuning; all reported results come from them.
- **Critical success index (CSI):** of the area either model floods, the share both flood.
- **Cusecs:** cubic feet per second, the unit Chennai news uses for reservoir releases.
- **Pre-wet street:** low ground where the model already holds ≥ 5 cm before the rain.